

HelixOTA

HelixOTA

**Mises à jour universelles et découplées par voie hertzienne
— zéro brique par conception.**

Résumé

Helix OTA est un système universel de mises à jour par voie hertzienne (OTA) profondément découplé : un plan de contrôle Go associé à des agents clients par système d'exploitation, conçu pour déployer en toute sécurité des mises à jour progressives de micrologiciels et d'applications sur des flottes allant d'une seule carte à des millions d'appareils. Sa première cible est Android 15 sur l'Orange Pi 5 Max.

Description courte

Helix OTA est un système universel de mises à jour par voie hertzienne — un plan de contrôle Go associé à des agents clients par système d'exploitation — conçu pour garantir l'absence de corruption du système, la validation des téléchargements et des déploiements progressifs granulaires. Sa première cible est Android 15 sur l'Orange Pi 5 Max, avec des adaptateurs pour Linux et Windows prévus.

Description longue

Helix OTA est un système universel, générique et profondément découplé de mises à jour par voie hertzienne (OTA), construit autour d'une promesse intransigeante : une mise à jour ne doit jamais transformer un appareil fonctionnel en brique. Il se compose d'un **plan de contrôle** serveur Go, de **SDK/agents clients** par système d'exploitation et d'un **tableau de bord** de gestion, conçu dès l'origine pour être intégrable à *n'importe quel* système d'exploitation via des adaptateurs modulaires, plutôt que d'être réécrit de zéro pour chaque plateforme. La première cible

de déploiement est Android 15 (toutes variantes) sur l'Orange Pi 5 Max, où le pipeline de compilation génère des images de flashage accompagnées d'un fichier .zip OTA validé et de fichiers de hachage obligatoires, garantissant qu'aucun artefact ne parvient à un appareil sans empreinte vérifiable. Linux, Windows et d'autres systèmes d'exploitation figurent sur la feuille de route, partageant le même mécanisme d'adaptation, ne nécessitant qu'un adaptateur — et non une réécriture complète.

L'architecture repose sur des garanties strictes définies par les opérateurs, traitées comme des invariants architecturaux non négociables : absence totale de corruption du système, validation obligatoire de chaque artefact avant tout déploiement, déploiement progressif granulaire (total ou par paliers de 5/10/30...100 % avec contrôle d'arrêt et de reprise), observabilité complète de la flotte, et passage à l'échelle linéaire, d'une seule carte sur un banc d'essai à des millions d'appareils sur le terrain. L'architecture verrouillée associe les mises à jour natives A/B d'Android côté appareil — `update_engine` d'AOSP avec AVB/dm-verity et retour automatique en cas d'échec de démarrage — à un plan de contrôle Go personnalisé et découplé, de sorte que la sécurité réside à la fois dans le chemin de démarrage proche du silicium *et* dans le serveur, et non dans une seule couche fragile. Deux interfaces sont délibérément conçues pour être extractibles : une interface d'adaptation au système d'exploitation, porteuse de la promesse d'une véritable universalité, et une interface de moteur de déploiement, rendant les campagnes progressives agnostiques au système d'exploitation. L'ensemble du système est décomposé en six sous-modules publics `ota-*` versionnés indépendamment — des blocs de construction réutilisables plutôt qu'un monolithe.

Helix OTA est actuellement en phase de spécification, de recherche et de développement de la couverture de tests. Le dépôt contient le corpus de conception officiel, le pipeline d'exportation de la documentation et l'échafaudage des sous-modules. Conformément à sa gouvernance anti-bluffer, il est explicitement indiqué qu'un serveur et un agent de production finis n'existent pas encore. Ce qui est livré aujourd'hui est le plan directeur et son échafaudage, présenté honnêtement comme tel.

Pourquoi nous l'avons conçu

OTA est généralement réinventé pour chaque appareil et chaque système d'exploitation, et une mauvaise mise à jour peut rendre une flotte entière inutilisable. Helix OTA a été conçu comme un système de mise à jour universel, axé sur la sécurité, que tout système d'exploitation peut adopter via des adaptateurs, avec des garanties de restauration et de validation intégrées à l'architecture plutôt que greffées après coup.

Pourquoi c'est un changement radical

Il refuse de considérer « ne jamais rendre un appareil inutilisable » et « déployer progressivement et de manière observable » comme des fonctionnalités au mieux de leurs capacités, que l'on espère tenir sous charge - ce sont des invariants architecturaux intégrés aussi bien dans le chemin de démarrage que dans le plan de contrôle. Et en faisant du moteur de déploiement et de la couche système des interfaces interchangeables plutôt que des hypothèses figées, le même plan de contrôle peut piloter Android aujourd'hui et être prêt à gérer d'autres systèmes d'exploitation demain, simplement en ajoutant un adaptateur - sans fork, sans réécriture, sans réinvention des garanties de sécurité auxquelles vous faites déjà confiance.

Ce qui est innovant

- **Deux interfaces extractibles** - une interface adaptateur-OS et un moteur de déploiement agnostique - transformant « universel » d'un mot marketing en une propriété structurelle du code.
- **Sécurité en profondeur** : A/B natif côté appareil (update_engine) + AVB/dm-verity + restauration automatique en cas d'échec du démarrage, le tout *superposé* à une validation des artefacts côté serveur - une mise à jour doit franchir plusieurs barrières indépendantes avant de pouvoir être persistée.
- **Décomposition catalogue-first, découplée** en six sous-modules ota-* réutilisables et versionnés indépendamment, que vous pouvez consommer à la carte plutôt que d'avaler un monolithe.
- **Transport principal HTTP/3 (QUIC)** avec repli automatique sur HTTP/2 et compression négociée Brotli/gzip - une diffusion moderne et à faible latence qui dégrade élégamment au lieu d'échouer.
- **Ingénierie anti-bullshit** : la conception et le statut sont explicitement marqués comme étant en phase de spécification, et rien de non construit n'est jamais présenté comme livré - l'honnêteté est érigée en valeur d'ingénierie de premier plan, et non reléguée à une clause de non-responsabilité en petits caractères.

Principaux défis techniques et comment nous les avons résolus

- **Garantir qu'une mauvaise mise à jour ne rende jamais un appareil inutilisable** - la promesse la plus difficile en matière de OTA. Résolu en imposant le A/B natif côté appareil sous

Android : `update_engine` écrit sur le slot inactif tandis que le slot actif continue de fonctionner, AVB/dm-verity vérifie cryptographiquement la chaîne de démarrage, et si le nouveau slot échoue à démarrer, l'appareil revient automatiquement en arrière – le tout sécurisé par une validation obligatoire des artefacts avant déploiement, de sorte qu'une charge corrompue est interceptée avant même de quitter le serveur.

- **Un seul système, plusieurs systèmes d'exploitation** – résolu en refusant d'intégrer des hypothèses Android au cœur du système. Une interface adaptateur-OS interchangeable isole les spécificités de la plateforme, et un moteur de déploiement agnostique maintient la logique des campagnes portable, chacun étant conservé dans un sous-module distinct afin qu'un nouveau système d'exploitation soit une addition, jamais une refonte complète.
- **Déploiements progressifs et interruptibles** – résolu avec un moteur de déploiement dédié qui raisonne en cohortes de pourcentage avec des seuils de succès/erreur et un contrôle explicite d'arrêt/avancement, délibérément découplé de HTTP pour que le même moteur puisse piloter des campagnes indépendamment du transport.

Pile technologique

- **Go + Gin** — choisi pour son modèle de concurrence et son empreinte de déploiement légère ; alimente le plan de contrôle, le moteur de déploiement et les validateurs d'artefacts, exposant la surface principale REST `/api/v1`.
- **Kotlin/KMP** — choisi afin que l'agent OTA Android embarqué puisse partager la logique entre les cibles ; gère l'intégralité de la boucle sur l'appareil : interrogation / téléchargement / vérification / application / rapport.
- **HTTP/3 (QUIC) → HTTP/2** — QUIC retenu comme transport principal pour une diffusion à faible latence et résiliente sur des liaisons mobiles instables, avec un repli automatique sur HTTP/2 afin qu'aucun appareil ne soit laissé de côté ; **Brotli/gzip** négocié à chaque requête pour réduire la taille des charges utiles.
- **PostgreSQL** — choisi pour l'intégrité relationnelle entre le registre des appareils, les campagnes et la télémétrie, où la précision de l'état de la flotte prime sur la vitesse brute d'écriture.
- **MinIO / S3** — retenu comme stockage d'artefacts afin que les images volumineuses de micrologiciels résident dans un stockage d'objets standard, découplé de la couche relationnelle.
- **AOSP update_engine + AVB/dm-verity + boot_control** — choisi car réutiliser les mécanismes éprouvés d'Android pour le Virtual A/B et le démarrage vérifié est plus sûr que de concevoir un système de mise à jour sur mesure ; utilisé pour

gérer les permutations de slots et la vérification cryptographique du démarrage sur l'appareil.

- **React** — choisi pour le tableau de bord de gestion où les opérateurs se connectent, téléversent des artefacts, pilotent les déploiements et surveillent l'état de la flotte en un seul endroit.
- **OpenTelemetry + Prometheus/Grafana** — retenu pour une instrumentation neutre vis-à-vis des fournisseurs ; utilisé pour rendre chaque étape d'un déploiement observable via des métriques et des tableaux de bord, plutôt que de reposer sur des suppositions.

État et notes de transparence

- **État : en développement.** Conformément à la gouvernance anti-bullshit du projet, il n'existe **aucun serveur ni agent de production fonctionnel à ce stade** — il s'agit d'une phase de spécification, de recherche et de développement de la couverture de tests. Le dépôt contient le corpus de conception officiel, le pipeline d'exportation de la documentation et l'échafaudage des sous-modules.
- Les six sous-modules publics réutilisables (ota-protocol, ota-artifact-validator, ota-rollout-engine, ota-update-engine-bridge, ota-android-agent, ota-telemetry-schema) sont hébergés sous github.com/HelixDevelopment/.
- Les chiffres de couverture de tests et de latence présents dans le dépôt constituent le registre en cours du projet, et n'ont pas été confirmés de manière indépendante. Les numéros de clauses HelixConstitution cités dans le README sont NON VÉRIFIÉS.
- **Licence : Apache-2.0.**

Niveau de priorité : Helix-primaire.